

Resampling & Monte Carlo Methods

Block 4.1: Data Science, Lecture 3

Oliver Lindemann

Erasmus University Rotterdam

1

Randomness

How can randomness help us to understand the world?

Random number generation is a tool in statistics and data science.

- resampling techniques
- understand the variability inherent in our analyses
- simulate data

2

Techniques that make use of Randomness

1. Resampling / Bootstrapping

- Repeated drawing or rearranging of samples from existing data to assess statistical estimates and their uncertainty.
- used e.g., assess variability analysis models

2. Monte Carlo Simulation / Data Simulation

- artificial generation of data (simulation)
- repeated random simulation to estimate probabilities

3. Random walk

A process where each step is random, often used to model movement or changes over time. Used for modelling cognitive processes like decision making, in psychology.

3

Bootstrapping

4

Example

Data: Reaction times in a verbal fluency task. $N=10$ participants, each participant has a mean reaction time.

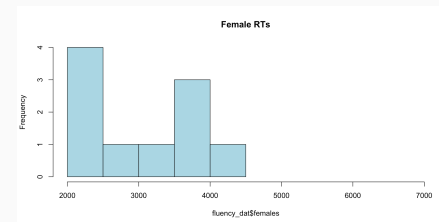
► Code

	females	males		Females	Males
1	2215	5036			
2	4063	4125	mean	3001	4396.4
3	3776	3900			
4	3567	4954	median	2851	4212
5	2346	3464			
6	3144	4299	sd	701.7	1133.2
7	2492	3182			
8	3576	6949			
9	2558	4857			
10	2273	3198			

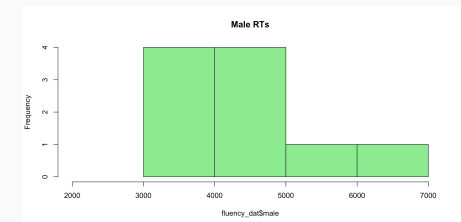
5

Data

```
1 hist(flucency_dat$females, xlim=c(2000, 7000), col="lightblue")
```



```
1 hist(flucency_dat$male, xlim=c(2000, 7000), col="lightgreen")
```



6

Excursus Statistics

7

Sample vs Population

How precise does the *mean of a sample* reflect the *mean of the population*?

What is the average error that we make if we estimate the *population mean*?

- **Sampling Distribution**
 - distribution of a statistical parameter (e.g., mean) obtained from repeated sampling of a population
- **Central Limit Theorem:**
 - With a large enough sample size, the sampling distribution of the **sample mean** is approximately normal, regardless of the population's distribution

8

Sampling Distribution, SE & CI

Standard Error of the Mean

The variability of the sample means and can be calculated:

$$SE = \frac{SD}{\sqrt{N}}$$

Confidence Interval

CI is a range of values where the 95% of the statistical estimates will fall.

$$CI_{\alpha} = M \pm Z_{\alpha/2} * SE$$

Problem

However, the central limit theorem counts only for the uncertainty of the *mean*. It does not account for the uncertainty of other statistics, like the *median*, or the *variance*.

9

Let's make the sampling distribution for this experiment

- **Sampling distribution:** the distribution of the parameter, if we sample over and over again
- we could
 1. Run the experiment 1,000 times
 2. calculate the mean (each time)
 3. Plot the distribution of those means

Alternative: Bootstrapping

Instead of running the experiment 1,000 times, we can use **resampling** to estimate the sampling distribution of the mean.

What is the best “simulation” of re-running the experiment?

- randomly sampling from the data that we have
- **resampling** to estimate the sampling distribution of the mean.

10

Bootstrapping: Practical example



11

Practical example: Bootstrapping

As an example, we resample one times from the female data

1. **Resampling:** Draw 10 random samples (with replacement) from the female data

```
1 resampled_dat = sample(fluency_dat$females, size = 10, replace = TRUE)
```

```
[1] 2346 2492 3576 2558 2273 2492 2558 2558 3776 3567
```

2. **Calculation parameter θ :** Calculate the mean of the resampled data

```
1 m_sample = mean(resampled_dat)
```

3. Display the mean of the resampled data

```
1 print(m_sample)
```

```
[1] 2819.6
```

12

Practical example: Bootstrapping

Means of 12 resampled data sets

► Code

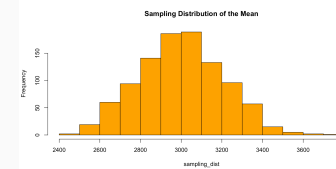
```
[1] "Sample 1: 3075.1"
[1] "Sample 2: 3007.7"
[1] "Sample 3: 3007"
[1] "Sample 4: 3066.4"
[1] "Sample 5: 2771.2"
[1] "Sample 6: 2737.3"
[1] "Sample 7: 2992.4"
[1] "Sample 8: 3168.6"
[1] "Sample 9: 3187.5"
[1] "Sample 10: 3015.1"
[1] "Sample 11: 3124"
[1] "Sample 12: 3383.9"
```

13

Bootstrapping: Practical example

1,000 resamples of the data and calculate the mean of each resample.

```
1 B = 1000 # Number of bootstrap iterations
2 sampling_dist = c()
3 for (i in 1:B) {
4   # 1. Resample
5   resampled_dat = sample(fluency_dat$females, size = 10, replace = TRUE)
6   # 2. Calculate the mean
7   m_sample = mean(resampled_dat)
8   # append to sampling distribution
9   sampling_dist = c(sampling_dist, mean(resampled_dat))
10 }
```



```
1 mean(sampling_dist) [1] 2999.6
```

```
1 sd(sampling_dist) [1] 206.5791
```

14

Short code for Resampling

The code from the previous slide can be shortened using the `replicate()` function.

```
1 B = 1000 # Number of bootstrap iterations
2 sampling_dist <- replicate(
3   B,
4   mean(sample(dat$females, size = 10, replace = TRUE))
5 )
```

15

Resampling: Practical example

Standard Error

Std. dev. of the sampling distribution

```
1 sd(sampling_dist)
```

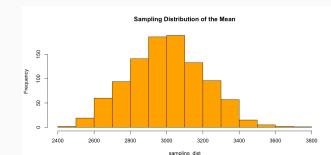
```
[1] 206.5791
```

Confidence Interval

The 95% range of the sampling distribution.

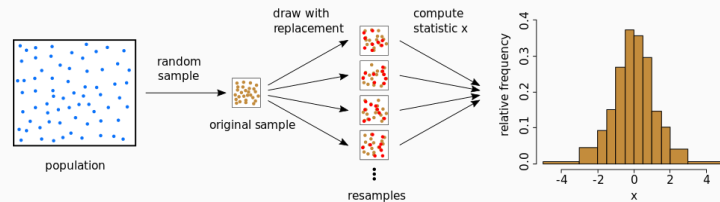
```
1 quantile(sampling_dist, probs = c(0.025, 0.975))
```

```
2.5%    97.5%
2608.715 3382.570
```



16

Summary: Principles of Bootstrapping



17

Summary: Bootstrap

- Repeated sampling from existing data (with replacement)
- Estimates the sampling and parameter distribution
- Method to estimate unknown population parameters and their uncertainty
- The technique has been introduced by Bradley Efron in 1979 and has become a standard tool in statistics & data science
- Useful for confidence intervals, population variances and biases

Advantages of Bootstrap methods

- simple to implement
- reduces dependencies on assumptions
- work effectively with small sample sizes
- supports complex data analysis

18

Further Resampling Methods

Techniques that fall under the umbrella term resampling:

- **Bootstrap**
 - Randomly creates many samples with replacement and recalculates the statistic.
- **Jackknife**
 - Systematically removes one observation at a time and recalculates the statistic.
- **Permutation testing**
 - A method where labels or values are randomly shuffled many times to see how unusual the observed result is compared to chance.
 - Permutation tests estimate a null distribution.
- **Cross-validation**
 - A method for evaluating a model by splitting data into parts, training on some parts, and testing on the others, repeated several times.

19

Bootstrapping Parameters of Statistical Models

Example: Linear Regression

The data set `mtcars` contains data on 32 cars, including their miles per gallon (mpg), weight (wt), and horsepower (hp).

```
1 data(mtcars)
2 print(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3

20

Linear Regression Model

Predicting consumption of fuel (miles per gallon, mpg) based on weight (wt) and horsepower (hp):

$$\text{mpg} = \beta_0 + \beta_1 \text{wt} + \beta_2 \text{hp} + \epsilon$$

```
1 model <- lm(mpg ~ wt + hp, data = mtcars) # Fit a linear regression model
2 summary(model) # Summary of the model
```

Call:
lm(formula = mpg ~ wt + hp, data = mtcars)

Residuals:

	Min	1Q	Median	3Q	Max
	-3.941	-1.600	-0.182	1.050	5.854

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	37.22727	1.59879	23.285	< 2e-16 ***
wt	-3.87783	0.63273	-6.129	1.12e-06 ***
hp	-0.03177	0.00903	-3.519	0.00145 **

21

Bootstrapping the Coefficients of the Linear Regression Model

Bootstrap distributions of the regression coefficients *wt* & *hp* (*boot_wt*, *boot_hp*)

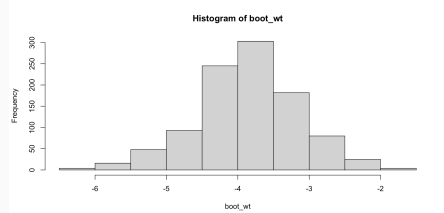
```
1 B <- 1000 # Number of bootstrap iterations
2
3 boot_wt = c()
4 boot_hp = c()
5
6 ## Non-parametric bootstrap
7 for (i in 1:B) {
8   # Resample rows with replacement
9   resampled_data <- mtcars[sample(nrow(mtcars), replace = TRUE), ]
10
11   # Fit the model to the resampled data
12   boot_model <- lm(mpg ~ wt + hp, data = resampled_data)
13
14   # Store coefficients
15   boot_wt <- c(boot_wt, coef(boot_model)["wt"])
16   boot_hp <- c(boot_hp, coef(boot_model)["hp"])
17 }
```

22

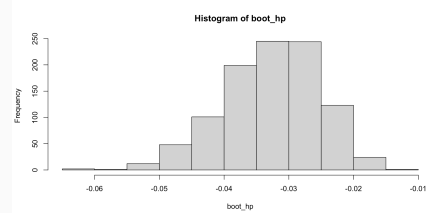
Bootstrapping the Coefficients of the Linear Regression Model

Bootstrap distributions of the regression coefficients *wt* & *hp* (*boot_wt*, *boot_hp*)

```
1 hist(boot_wt)
```



```
1 hist(boot_hp)
```



23

Bootstrapping the Coefficients of the Linear Regression Model

Calculate standard errors

```
1 sd(boot_wt)
```

```
[1] 0.7167332
```

```
1 sd(boot_hp)
```

```
[1] 0.007426349
```

Calculate 95% confidence intervals

using the `quantile()` function

```
1 quantile(boot_wt, probs = c(0.025, 0.975))
```

```
2.5%      97.5%
-5.369070 -2.430203
```

```
1 quantile(boot_hp, probs = c(0.025, 0.975))
```

```
2.5%      97.5%
-0.04840852 -0.02011802
```

24

Monte Carlo Methods & Data Simulation

25

Monte Carlo Simulation

- simulating data using random number generation
- known as Monte Carlo simulation, named after the famous casino in Monaco
- used to estimate probabilities and distributions of statistical parameters

26

Pseudo-Random Number Generator (RNG)

... Generating 1 random numbers between 1 and 10:

```
1 runif(n=1, min = 1, max = 10)
```

```
[1] 5.551088
```

```
1 runif(n=15, min = 1, max = 10)
```

```
[1] 4.809874 5.994477 3.613613 9.334933 5.137996 1.217823 6.939371 6.432950
```

```
[9] 5.378063 1.502514 3.417463 1.443256 2.151619 8.250110 4.754646
```

Draw 15 random numbers from a distribution with a mean of $M = 4$ and $SD = 2$

```
1 rnorm(n=15, mean = 4, sd = 2)
```

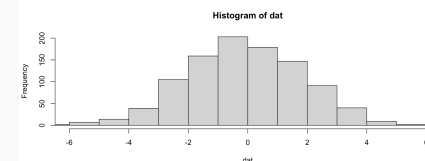
```
[1] 4.607066 2.050927 3.072124 5.769400 3.800706 2.615094 3.942358 4.767152
```

```
[9] 4.050217 4.945882 4.975085 4.243147 2.175294 2.444942 4.742235
```

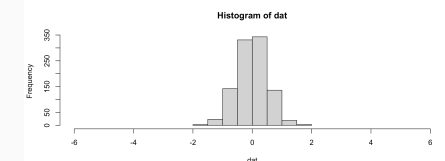
27

Comparing Random Distributions

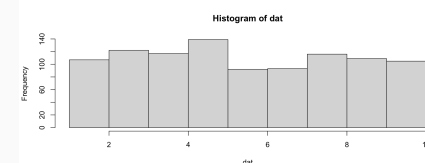
```
1 dat = rnorm(n=1000, mean = 0, sd = 2)
2 hist(dat, xlim=c(-6, 6))
```



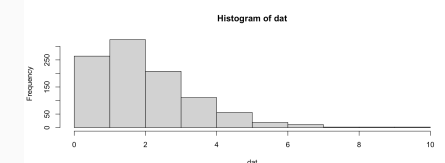
```
1 dat = rnorm(n=1000, mean = 0, sd = 0.5)
2 hist(dat, xlim=c(-6, 6))
```



```
1 dat = runif(n=1000, min = 1, max = 10)
2 hist(dat)
```



```
1 dat = rgamma(1000, shape=2, rate=1)
2 hist(dat)
```



Seed numbers in RNG

- computer generated random numbers are not truly random, but **pseudo-random**.
- A seed is a starting number a computer uses to kick off its procedure for making a list of numbers that look random.
- “Seed” & reproducibility
 - Using the same seed produces the same list of pseudo-random numbers
 - experiments or simulations can be repeated identically

```
1 set.seed(67) # any number
```

Example: *t*-test

t-test: Compare two groups of data, e.g., reaction times in a verbal fluency task.

```
1 print(fluency_dat)
```

	females	males
1	2215	5036
2	4063	4125
3	3776	3900
4	3567	4954
5	2346	3464
6	3144	4299
7	2492	3182
8	3576	6949
9	2558	4857
10	2273	3198

```
1 mean(fluency_dat$females)
```

```
[1] 3001
```

```
1 mean(fluency_dat$males)
```

```
[1] 4396.4
```

```
1 mean(fluency_dat$females) -  
2 mean(fluency_dat$males)
```

```
[1] -1395.4
```

```
1 sd(fluency_dat$females)
```

```
[1] 701.7466
```

```
1 sd(fluency_dat$males)
```

```
[1] 1133.225
```

Example: *t*-test

```
1 t.test(fluency_dat$females, fluency_dat$males, var.equal = TRUE)
```

Two Sample t-test

```
data: fluency_dat$females and fluency_dat$males  
t = -3.3105, df = 18, p-value = 0.00389  
alternative hypothesis: true difference in means is not equal to 0  
95 percent confidence interval:  
-2280.9446 -509.8554  
sample estimates:  
mean of x mean of y  
3001.0 4396.4
```

What is statistical power of this *t*-test?

- The probability of finding a significant result, if there is indeed a difference in the population.
- The likelihood to observe a significant result, if the study is repeated or simulated over and over again.

Simulation-based Power Analysis

Principle: Simulate data and count how often we find a significant effect

1. Simulate data

```
1 n = 10 # sample size per group
2 sim_dat = data.frame(
3   females = rnorm(n = n, mean = 3001, sd = 702),
4   males   = rnorm(n = n, mean = 4396, sd = 1133)
5 )
```

```
   females   males
1 3909.769 5458.740
2 3256.608 5482.469
3 3117.153 4138.116
4 3184.049 5006.123
5 3319.424 3863.864
6 3826.364 5304.615
7 1969.931 4129.839
8 3067.630 7502.088
9 3296.114 2217.913
10 1544.725 4671.545
```

32

Simulation-based Power Analysis

2. Run a *t*-test

Compare the two groups, Males and Females. The result of the *t*-test is stored in `res`.

```
1 res = t.test(sim_dat$females, sim_dat$males, var.equal = TRUE)
```

3. Check significance

Check if the $p \leq .05$ (alpha level = .05)

```
1 res$p.value <= 0.05
```

```
[1] TRUE
```

... next simulation

33

Simulation-based Power Analysis

```
1 n = 10
2 B = 1000 # number of simulations
3 results = c()
4
5 for (i in 1:B) {
6   sim_dat = data.frame( # 1. simulation
7     females = rnorm(n = n, mean = 3001, sd = 702),
8     males   = rnorm(n = n, mean = 4396, sd = 1133)
9   )
10  res = t.test(sim_dat$females, sim_dat$males, var.equal = TRUE) # 2. t-test
11
12  is_sig = res$p.value <= 0.05 # 3. check significance
13  results = c(results, is_sig) # store
14 }
```

Count the number of significant results and calculate the proportion of significant results (i.e., the power).

```
1 sum(results) / length(results)
```

```
[1] 0.874
```

34

How robust is this kind of power analysis?

Just increase the number of simulations to 10,000 and re-run the code or rerun the analysis multiple times. The results should be very similar.

```
[1] 0.878
[1] 0.879
[1] 0.885
[1] 0.869
[1] 0.888
```

35

Bootstrapped Permutation Tests

36

Rationale of Statistic Tests

Can't we make use of randomness to make a statistical test?

Example: Difference between females and males

What is a p -value telling us?

- probability of observing a test statistic (e.g. mean difference) as extreme as (or more extreme than) the one observed, assuming the null hypothesis is true.
- $p = P(D|H_0)$

What does the p -value tell us in our example?

- Females and males stem from the same population.
- In other words, there is no difference between the two groups.

37

Permutation t -Tests

1. Pooled data

Combine the two groups into one pooled data set ("a single population")

```
1 pooled_data = c(fluency_dat$females, fluency_dat$males) # our "population"
```

2. Permute Data

Divide the "population" randomly into two new groups

```
1 shuffled_data = sample(pooled_data) # shuffle the data
2 random_females = shuffled_data[1:10] # first 10 values are the "new females"
3 random_males = shuffled_data[11:20] # the rest are the "new males"
```

3. Calculate the mean difference between the two new groups

```
1 mean_diff = mean(random_females) - mean(random_males)
```

38

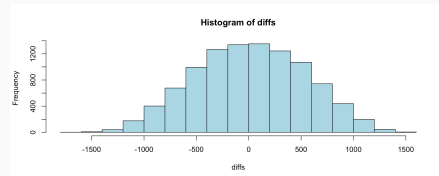
Permutation t -Tests

```
1 n = 10
2 pooled_data = c(fluency_dat$females, fluency_dat$males)
3
4 diffs = c()
5 for (i in 1:10000) {
6   shuffled_data = sample(pooled_data) # 1. permute data
7   random_females = shuffled_data[1:10]
8   random_males = shuffled_data[11:20]
9
10  d = mean(random_females) - mean(random_males) # 2. calculate mean difference
11  diffs = c(diffs, d) # 3. store
12 }
```

39

Results of the Permutation Test

```
1 hist(diffs, col="lightblue")
```



Calculate p -values

The observed mean difference between females and males was -1395.4

```
1 d_obs = -1395.5 # observed difference
2 is_extreme = abs(diffs) >= abs(d_obs) # is it more extreme than d_obs
3 sum(is_extreme) / length(diffs) # p-value
```

```
[1] 0.0015
```

40

Summary

- **Bootstrapping** is a resampling technique that allows us to estimate the sampling distribution of a statistic by repeatedly resampling with replacement from the observed data.
- **Monte Carlo Simulation** is a method that uses random sampling to estimate probabilities and distributions of statistical parameters.
- Both techniques rely on randomness to provide insights into the variability and uncertainty of statistical estimates, making them powerful tools in data analysis and statistical inference.
- **Permutation tests** are a type of non-parametric test that involves reshuffling the data to assess the significance of a result.

41

Advantages of Resampling and Monte Carlo Simulation Methods

Resampling methods such as bootstrapping and permutation tests and Monte Carlo simulation methods, offer several advantages over traditional parametric approaches:

- **Non-parametric methods, no assumptions:** These methods do not require any parametric assumptions about the underlying distributions (normality, homoscedasticity, etc.).
- **Flexibility:** Resampling methods can be applied to a wide range of statistical problems
- **Ease of Implementation:** These methods are relatively easy to implement using modern computing tools and software.
- **Intuitive Understanding:** The concepts behind resampling are often more intuitive than traditional parametric approaches.

Exercises in the tutorial group next week will repeat all analyses and simulations from this lecture.

42

Questions?

43